Il branch-and-bound basato sulla programmazione lineare (1/2)

Indice

Perché un algoritmo branch-and-bound?

L'idea in parole e in una figura

Definizioni e notazione

L'algoritmo

Breve riepilogo dell'algoritmo del simplesso duale

Strategie di selezione della variabile di branching

Strategie di selezione del nodo di branching

Obiettivi

Alla fine di questo capitolo sarai in grado di:

- spiegare come le operazioni di branching dividono un problema PLI in problemi LP più piccoli, e perché nessuna soluzione ammissibile viene persa;
- applicare i tre criteri di potatura, e decidere quando il bound di un nodo si può arrotondare per difetto;
- eseguire a mano un algoritmo branch-and-bound basato sul rilassamento lineare, riottimizzando ogni nodo con l'algoritmo del simplesso duale;
- calcolare il lower bound, l'upper bound e il gap di ottimalità a ogni iterazione;
- confrontare diverse strategie di selezione della variabile di branching e del nodo di branching sulla stessa istanza.

Prerequisiti

- Dal modulo *Programmazione lineare*, Capitolo 10 «Algoritmo del simplesso duale»: dizionari del simplesso, basi duali ammissibili e operazioni di pivot duali (un breve riepilogo è dato in questo capitolo)

- Da questo modulo, il capitolo «Programmazione lineare intera»: problemi PLI, soluzioni ammissibili e ottime, e l'esempio dell'auto

- Da questo modulo, il capitolo «Bound primali e duali»: il rilassamento lineare, i lower bound dalle soluzioni ammissibili, gli upper bound dal rilassamento lineare e il gap di ottimalità

Perché un algoritmo branch-and-bound?

Introduzione: Perché un algoritmo branch-and-bound?

- Ripartiamo dall'esempio dell'auto, l'esempio guida del modulo
- Il suo rilassamento lineare dà una soluzione frazionaria e dei bound su $z(\mathrm{ILP})$, ma non una soluzione ottima
- Elencare tutti i punti interi qui funziona, ma non sui problemi più grandi
- Serve un metodo che **trovi** una soluzione ottima e ne **dimostri** l'ottimalità risolvendo solo pochi problemi LP

L'esempio dell'auto

- Partiamo dall'**esempio dell'auto**: x_1 e x_2 sono i numeri di auto di due tipi
- Il problema PLI è il seguente:

$$
\begin{array}{rrcrcr}
\max & 3\,x_1 & + & 2\,x_2 & & \\
\text{soggetto a} & 10\,x_1 & + & 4\,x_2 & \leq & 45 \\
 & 4\,x_1 & + & 14\,x_2 & \leq & 49 \\
 & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0}
\end{array}
$$

Il rilassamento lineare dell'esempio dell'auto

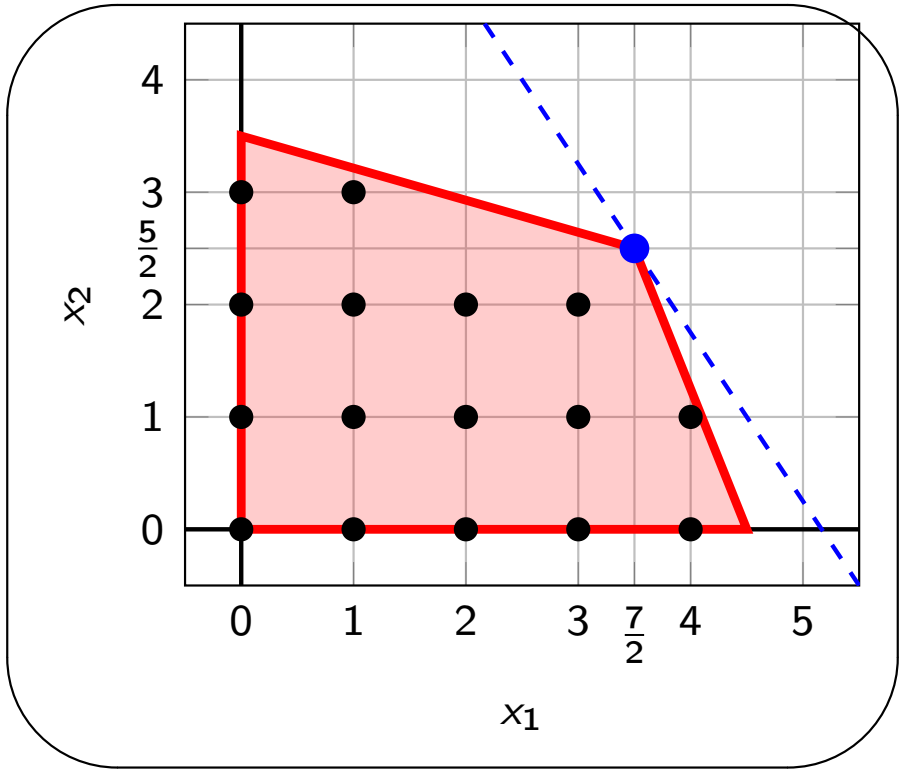
- La soluzione ottima del rilassamento lineare è frazionaria:

$$\tilde{x}_1 = \frac{7}{2}, \quad \tilde{x}_2 = \frac{5}{2}, \qquad z(\mathrm{LP}) = \frac{31}{2}.$$

- Arrotondandola per difetto otteniamo la soluzione ammissibile $\bar{\boldsymbol{x}} = (3, 2)$ di valore 13
- Poiché i costi sono interi, il rilassamento lineare dà anche l'upper bound $\lfloor z(\mathrm{LP}) \rfloor = 15$:

$$13 \quad \leq \quad z(\mathrm{ILP}) \quad \leq \quad 15.$$

I punti interi dell'esempio dell'auto



La regione ammissibile del rilassamento lineare (in rosso), i suoi 16 punti interi (in nero) e la soluzione ottima del rilassamento lineare (in blu), con la retta dell'obiettivo che passa per essa (tratteggiata).

L'enumerazione non scala

- La regione ammissibile contiene 16 punti interi
- Enumerarli tutti qui è ancora possibile, ma il numero di punti interi cresce esponenzialmente con il numero di variabili
- Serve un modo per escludere interi gruppi di punti interi senza visitarli

Una sfida

Prima di proseguire, prova a rispondere a due domande.

Quale soluzione ammissibile è **ottima**?

Come possiamo dimostrare che non esiste una soluzione migliore,
risolvendo solo pochi problemi LP
e senza elencare tutti i punti interi?

- Gli esempi svolti di questo capitolo rispondono a entrambe le domande

Riepilogo

- Il rilassamento lineare dell'esempio dell'auto ha la soluzione ottima frazionaria $(\frac{7}{2}, \frac{5}{2})$, con $z(\mathrm{LP}) = \frac{31}{2}$
- L'arrotondamento e il rilassamento lineare danno $13 \leq z(\mathrm{ILP}) \leq 15$, ma nessun certificato di ottimalità
- L'enumerazione completa dei punti interi non scala
- L'algoritmo branch-and-bound chiude il gap risolvendo una sequenza di problemi LP

L'idea in parole e in una figura

Introduzione: L'idea in parole e in una figura

- Prima delle definizioni formali, vediamo l'idea dell'algoritmo sull'esempio dell'auto
- Due operazioni: il **branching** divide il problema, il **bounding** decide quali parti vanno ancora esplorate
- Una sola figura le mostra entrambe

Il branch-and-bound basato sul rilassamento lineare

- L'**algoritmo branch-and-bound basato sul rilassamento lineare** (in breve, **algoritmo B&B**) è un metodo esatto per i problemi PLI
- Si dice **basato sul rilassamento lineare** perché risolve rilassamenti lineari per guidare la ricerca di una soluzione ottima

Operazioni di branching

- Nella soluzione ottima del rilassamento lineare dell'esempio dell'auto la variabile x_1 assume un valore frazionario compreso tra 3 e 4
- Ogni soluzione ammissibile del problema PLI soddisfa $x_1 \leq 3$ oppure $x_1 \geq 4$
- Quindi possiamo dividere il problema in due problemi LP più piccoli, uno per ciascun lato, ed eliminare la striscia intermedia, che non contiene punti interi:

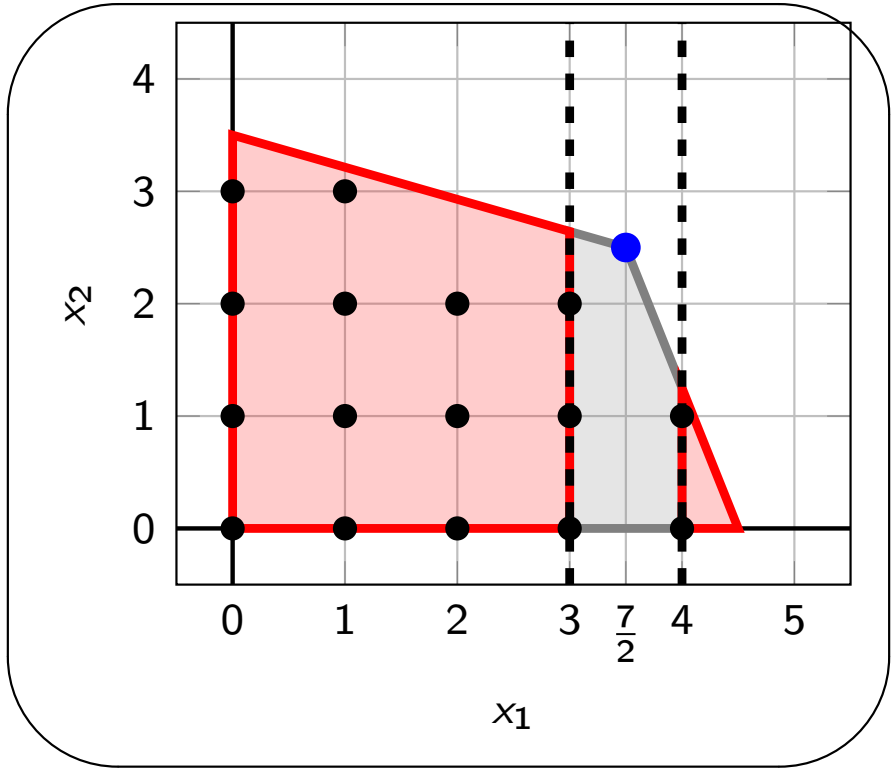
$$\rightsquigarrow \textbf{operazioni di branching}.$$

Operazioni di bounding

- Ogni nuovo problema LP viene risolto
- Il suo valore ottimo limita dall'alto il valore di ogni soluzione nella sua regione
- Quindi ci dice se la regione va esplorata ancora o si può scartare:

$$\rightsquigarrow \textbf{operazioni di bounding}.$$

Branching su x_1 alla radice



Le due regioni rosse sono le regioni ammissibili dei figli ($x_1 \leq 3$ e $x_1 \geq 4$); la striscia grigia $3 < x_1 < 4$ viene tagliata via, e non contiene punti interi.

Enumerazione implicita

Un algoritmo branch-and-bound esplora implicitamente tutte le soluzioni ammissibili:

divide la regione ammissibile in regioni più piccole (**branching**)

e scarta ogni regione che non può contenere una soluzione migliore della migliore trovata finora (**bounding**).

Riepilogo

- L'algoritmo B&B è un metodo esatto per i problemi PLI, guidato dai rilassamenti lineari
- **Branching**: un valore frazionario $3 < \tilde{x}_1 < 4$ divide il problema in $x_1 \leq 3$ e $x_1 \geq 4$; la striscia intermedia non contiene punti interi
- **Bounding**: il valore ottimo di ogni nuovo problema LP dice se la sua regione va esplorata ancora
- Tutte le soluzioni ammissibili vengono esplorate **implicitamente**

Definizioni e notazione

Introduzione: Definizioni e notazione

- Fissiamo il problema: un problema PLI in forma canonica di massimo con dati interi
- Definiamo gli oggetti dell'algoritmo: l'albero branch-and-bound, la soluzione incumbent, le operazioni di branching e di bounding, la lista dei problemi LP
- Definiamo il lower bound, l'upper bound e il gap di ottimalità, che misurano i progressi della ricerca

Problemi PLI in forma canonica di massimo

- Dati n valori $c_j \in \mathbb{Z}$, $m \cdot n$ valori $a_{ij} \in \mathbb{Z}$ e m valori $b_i \in \mathbb{Z}$, un problema di **programmazione lineare intera** (PLI) in forma canonica di massimo, con n variabili intere e dati interi, si scrive:

$$\max \quad \sum_{j=1}^{n} c_j x_j$$

$$\text{soggetto a} \quad \sum_{j=1}^{n} a_{ij} x_j \leq b_i, \qquad \forall i \in \{1, 2, \ldots, m\},$$

$$x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \ldots, n\}.$$

Notazione

- L'esempio dell'auto è un problema PLI con $n = 2$ e $m = 2$
- Indichiamo con $z(\mathrm{ILP})$ il valore ottimo del problema PLI, e con $\tilde{\boldsymbol{x}}(\mathrm{ILP})$ una soluzione ottima
- Il **rilassamento lineare** del problema PLI, detto problema LP_1, si ottiene sostituendo i vincoli di integralità $x_j \in \mathbb{Z}_{\geq 0}$ con i vincoli di non negatività $x_j \geq 0$, per ogni $j \in \{1, 2, \ldots, n\}$

L'albero branch-and-bound

Le **operazioni di branching** generano
un **albero branch-and-bound** che contiene problemi LP.

Ogni problema LP si ottiene dal rilassamento lineare
aggiungendo opportuni **vincoli di branching**.

- La struttura ricorsiva di un algoritmo B&B induce l'albero
- Ogni **nodo di branching** dell'albero corrisponde a un problema LP; la radice è il problema LP_1

Definizione 1: indice del nodo di branching

Per ogni nodo di branching k, il problema LP associato si indica con LP_k. Se LP_k ha una soluzione ottima, questa si indica con $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ e il suo valore ottimo con $z(\mathrm{LP}_k)$.

*L'intero k si chiama **indice del nodo di branching**: identifica il nodo in modo univoco, e i nodi sono numerati nell'ordine in cui vengono risolti i loro problemi LP.*

Soluzione incumbent e lower bound

- Un algoritmo B&B conserva la migliore soluzione ammissibile del problema PLI trovata durante la ricerca
- Questa soluzione si chiama **soluzione incumbent** e si indica con $\bar{\boldsymbol{x}}$; il suo valore è un **lower bound** LB su $z(\mathrm{ILP})$
- Prima di trovare una soluzione ammissibile poniamo $\mathrm{LB} = -\infty$
- Il lower bound e la soluzione incumbent vengono aggiornati ogni volta che si trova una soluzione ammissibile migliore. Alla terminazione abbiamo un **certificato algoritmico** che

$$z(\mathrm{ILP}) = \mathrm{LB} \qquad \text{e} \qquad \tilde{\boldsymbol{x}}(\mathrm{ILP}) = \bar{\boldsymbol{x}}.$$

Branching binario

- L'operazione di branching più comune è il **branching binario**
- Agisce su una sola **variabile di branching** e produce due nodi di branching figli:
  - uno con un **upper bound** aggiuntivo sulla variabile di branching
  - uno con un **lower bound** aggiuntivo sulla variabile di branching

Vincoli di branching verso il basso e verso l'alto

- Sia $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ una soluzione ottima del problema LP_k, e sia x_j, con $j \in \{1, 2, \ldots, p\}$, una variabile intera che assume un valore frazionario $\tilde{x}_j$ (le variabili intere sono $x_1, \ldots, x_p$; in un problema PLI $p = n$)
- Il branching binario su x_j genera due problemi LP figli:

1. il problema LP_k con il **vincolo di branching verso il basso** aggiuntivo
$$x_j \leq \lfloor \tilde{x}_j \rfloor ;$$
2. il problema LP_k con il **vincolo di branching verso l'alto** aggiuntivo
$$x_j \geq \lfloor \tilde{x}_j \rfloor + 1.$$

Correttezza del branching

- I due nodi figli si chiamano **nodo di branching verso il basso** e **nodo di branching verso l'alto**
- In entrambi $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ non è più ammissibile

Ogni soluzione ammissibile del problema PLI contenuta nella regione di LP_k soddisfa

$$x_j \leq \lfloor \tilde{x}_j \rfloor \quad \text{oppure} \quad x_j \geq \lfloor \tilde{x}_j \rfloor + 1,$$

quindi è contenuta nella regione di uno dei due figli.

Questo garantisce la correttezza dell'algoritmo B&B.

Operazioni di bounding

- Il valore $z(\mathrm{LP}_k)$ è un upper bound sul valore di ogni soluzione ammissibile del problema PLI nella regione di LP_k
- Quando tutti i valori sono interi, questo bound si può arrotondare per difetto

Osservazione 1: arrotondamento del bound di un nodo

Poiché tutte le variabili e tutti i costi sono interi, ogni soluzione ammissibile nella regione di LP_k *ha valore al più* $\lfloor z(\mathrm{LP}_k) \rfloor$.

Dimostrazione

- Poiché $c_j \in \mathbb{Z}$ per ogni j, il valore

$$\sum_{j=1}^{n} c_j \bar{x}_j$$

di una soluzione ammissibile $\bar{\boldsymbol{x}}$ è una somma di prodotti di interi, quindi un intero

- Poiché è al più $z(\mathrm{LP}_k)$, è al più il massimo intero non superiore a $z(\mathrm{LP}_k)$, cioè $\lfloor z(\mathrm{LP}_k) \rfloor$

□

Il bound di un nodo

- Di conseguenza, chiamiamo **bound del nodo** k il valore

$$\mathrm{UB}(\mathrm{LP}_k) = \begin{cases} \lfloor z(\mathrm{LP}_k) \rfloor & \text{se } p = n \text{ e } c_j \in \mathbb{Z} \\ & \text{per ogni } j \in \{1, 2, \ldots, n\}, \\ z(\mathrm{LP}_k) & \text{altrimenti.} \end{cases}$$

- Il bound si arrotonda per difetto solo quando tutte le variabili sono intere ($p = n$) e tutti i costi sono interi

Criteri di potatura (1/2)

- Dato un nodo di branching k, su di esso non si esegue alcuna operazione di branching se vale uno dei seguenti **criteri di potatura**. In questi casi si dice che il nodo è **potato**

1. **Potatura per inammissibilità**: il problema LP_k è inammissibile, quindi la sua regione non contiene soluzioni ammissibili del problema PLI
2. **Potatura per integralità**: le variabili intere hanno valori interi in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$, quindi $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ è ammissibile per il problema PLI, ed è ottima nella regione di LP_k. Se $z(\mathrm{LP}_k) > \mathrm{LB}$, il lower bound e la soluzione incumbent vengono aggiornati

Criteri di potatura (2/2)

3. **Potatura per bound**: il bound del nodo non supera il lower bound,

$$\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB},$$

quindi la regione di LP_k non contiene soluzioni migliori di quella incumbent

I criteri si verificano in quest'ordine:
inammissibilità, integralità, bound.

Quando valgono sia l'integralità sia il bound,
il nodo è potato per integralità.

La lista dei problemi LP

- Un nodo di branching che non viene potato è inserito in una **lista** L di problemi LP in attesa di un'operazione di branching
- L'ordine in cui i problemi LP vengono estratti dalla lista determina la **strategia di selezione del nodo di branching**
- La scelta della variabile di branching determina la **strategia di selezione della variabile di branching**
- Entrambe sono discusse dopo l'algoritmo

Upper bound

- Se la lista L non è vuota, ogni soluzione ammissibile migliore di quella incumbent si trova nella regione di qualche nodo della lista
- Di conseguenza, l'algoritmo B&B ha l'**upper bound**

$$z(\mathrm{ILP}) \quad \leq \quad \mathrm{UB} = \max_{k \in L} \{\mathrm{UB}(\mathrm{LP}_k)\}\,.$$

- Quando la lista è vuota, la soluzione incumbent è ottima e $\mathrm{UB} = \mathrm{LB}$

Gap di ottimalità

- L'upper bound si calcola dopo la soluzione della radice e si aggiorna alla fine di ogni iterazione
- All'algoritmo non serve, ma misura i progressi attraverso il **gap di ottimalità**:

$$\text{gap di ottimalità} = \frac{\text{UB} - \text{LB}}{|\text{LB}|}, \qquad \text{per LB} \neq 0.$$

Inizializzazione euristica

Prima dell'algoritmo B&B si può eseguire
un **algoritmo euristico** per trovare una buona soluzione ammissibile.

Il lower bound LB e la soluzione incumbent $\bar{\boldsymbol{x}}$
vengono allora inizializzati con questa soluzione.

- Un buon lower bound iniziale permette all'algoritmo di potare prima i nodi per bound, e può ridurre di molto la dimensione dell'albero branch-and-bound
- Se il lower bound iniziale soddisfa già $\mathrm{UB}(\mathrm{LP}_1) \leq \mathrm{LB}$, la radice è potata per bound: la soluzione euristica è ottima e non serve alcun branching

Riepilogo (1/2)

Problema e albero:

- Problema PLI in forma canonica di massimo con dati interi; la radice è il rilassamento lineare LP_1
- Ogni nodo k è un problema LP LP_k; i nodi sono numerati nell'ordine in cui vengono risolti

Branching:

- Su una variabile intera frazionaria: $x_j \leq \lfloor \tilde{x}_j \rfloor$ e $x_j \geq \lfloor \tilde{x}_j \rfloor + 1$
- Nessuna soluzione ammissibile del problema PLI viene persa

Riepilogo (2/2)

Bounding:

- Bound del nodo: $\mathrm{UB}(\mathrm{LP}_k) = \lfloor z(\mathrm{LP}_k) \rfloor$ quando tutte le variabili e tutti i costi sono interi
- Potatura per inammissibilità, per integralità, per bound ($\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB}$), in quest'ordine

Bound della ricerca:

- LB dalla soluzione incumbent, $\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k)$ dalla lista
- Gap di ottimalità $(\mathrm{UB} - \mathrm{LB})/|\mathrm{LB}|$; una soluzione euristica può inizializzare LB

L'algoritmo

Introduzione: L'algoritmo

- Mettiamo insieme le operazioni nello pseudocodice dell'algoritmo B&B e della sua sottoprocedura
- Vediamo la regola di salto, che evita di risolvere figli inutili, e come si numerano i nodi
- Contiamo i nodi di un albero completo: questo spiega perché la potatura è essenziale

L'algoritmo B&B (1/3)

Input: Un problema PLI in forma canonica di massimo con dati interi.

Output: Una soluzione ottima del problema PLI (se esiste) oppure un certificato di inammissibilità/illimitatezza.

Passo 0: **Inizializzazione**

0.1 Rilassare i vincoli di integralità e risolvere il problema LP_1.

1. se $\tilde{\boldsymbol{x}}(\mathrm{LP}_1)$ è **intera**, allora è una soluzione ottima e $z(\mathrm{LP}_1)$ è il valore ottimo del problema PLI. `STOP`.
2. se il problema LP_1 è **inammissibile**, allora il problema PLI è inammissibile. `STOP`.
3. se il problema LP_1 è **illimitato**, allora il problema PLI è inammissibile oppure illimitato. `STOP`.

L'algoritmo B&B (2/3)

Passo 0: Inizializzazione (continua)

0.2 Inserire il problema LP_1 in una **lista dei problemi LP per il branching** (indicata con L). Porre il **lower bound** $\mathrm{LB} \leftarrow -\infty$ (oppure il valore di una soluzione euristica), e porre un contatore dei nodi di branching $c \leftarrow 1$.

Passo 1: **Test di ottimalità**

0.1 se la lista L è vuota e $\mathrm{LB} \neq -\infty$, allora LB è il valore ottimo e la **soluzione incumbent** $\bar{\boldsymbol{x}}(\mathrm{ILP})$ è una soluzione ottima del problema PLI. `STOP`.

0.2 se la lista L è vuota e $\mathrm{LB} = -\infty$, allora il problema PLI è inammissibile. `STOP`.

Passo 2: **Strategia di selezione del nodo di branching**

Selezionare e rimuovere un problema LP_k dalla lista L.

L'algoritmo B&B (3/3)

Passo 3: **Strategia di selezione della variabile di branching**

Selezionare una variabile x_j con valore frazionario $\tilde{x}_j$ in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ e generare due nuovi problemi LP:

0.1 il problema $\mathrm{LP}_{c+1} \rightsquigarrow$ il problema LP_k con il **vincolo di branching verso il basso** aggiuntivo: $x_j \leq \lfloor \tilde{x}_j \rfloor$

0.2 il problema $\mathrm{LP}_{c+2} \rightsquigarrow$ il problema LP_k con il **vincolo di branching verso l'alto** aggiuntivo: $x_j \geq \lfloor \tilde{x}_j \rfloor + 1$

Eseguire la **Sottoprocedura di soluzione del nodo di branching** per il problema LP_{c+1} e per il problema LP_{c+2}; la seconda si salta se, dopo la prima, $\lfloor z(\mathrm{LP}_k) \rfloor \leq \mathrm{LB}$. Incrementare il contatore dei nodi di branching $c \leftarrow c + 2$ e andare al Passo 1.

Ogni figlio viene risolto e classificato
dalla **Sottoprocedura di soluzione del nodo di branching**:
questo è il suo pseudocodice.

Sottoprocedura di soluzione del nodo di branching

Input: Un problema LP_k associato a un nodo di branching.

Passo 1: **Operazioni di bounding**

Risolvere il problema LP_k.

0.1 se il problema LP_k è inammissibile, allora `STOP`.

0.2 se $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$ è intera, allora, se $z(\mathrm{LP}_k) > \mathrm{LB}$, aggiornare il **lower bound** e la **soluzione incumbent**:

$$\mathrm{LB} \leftarrow z(\mathrm{LP}_k) \quad \text{e} \quad \bar{\boldsymbol{x}}(\mathrm{ILP}) \leftarrow \tilde{\boldsymbol{x}}(\mathrm{LP}_k)$$

Rimuovere dalla lista L tutti i problemi LP con $\lfloor z(\mathrm{LP}) \rfloor \leq \mathrm{LB}$ e `STOP` (anche quando $z(\mathrm{LP}_k) \leq \mathrm{LB}$).

0.3 se $\lfloor z(\mathrm{LP}_k) \rfloor \leq \mathrm{LB}$, allora `STOP`.

Passo 2: **Inserimento nella lista**

Inserire il problema LP_k nella lista L e `STOP`.

Osservazioni sullo pseudocodice

- Il problema LP con il vincolo di branching verso il basso si chiama ***nodo di branching verso il basso***; il problema LP con il vincolo di branching verso l'alto si chiama ***nodo di branching verso l'alto***
- L'ordine in cui vengono risolti determina l'ordine in cui i problemi LP entrano nella lista L
- Lo pseudocodice non dice come si risolvono i problemi LP: si può usare qualunque metodo per i problemi LP
- Se si usa l'**algoritmo del simplesso duale**, ogni figlio si risolve a partire dal dizionario ottimo del padre: dopo l'aggiunta del vincolo di branching questo dizionario è ancora duale ammissibile, quindi in generale bastano poche operazioni di pivot (si veda il riepilogo dell'algoritmo del simplesso duale)

La regola di salto

- Il bound di un figlio non supera mai il bound del padre, perché il figlio ha un vincolo in più
- Di conseguenza, se dopo la soluzione del primo figlio abbiamo $\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB}$, il secondo figlio non può contenere una soluzione migliore, e **non viene risolto**
- Questa è la **regola di salto**

La numerazione dei nodi

- Un figlio saltato non riceve un indice di nodo
- I nodi sono numerati nell'ordine in cui vengono risolti i loro problemi LP, quindi gli indici dei nodi risolti sono sempre $1, 2, \ldots, c$
- L'ordine in cui vengono risolti i due figli determina anche l'ordine in cui entrano nella lista L
- In queste slide si risolve per primo il nodo di branching verso il basso, salvo indicazione contraria

Numero di nodi di branching

- Un albero branch-and-bound binario **completo** con $n \geq 1$ livelli ha 2^{n-1} nodi di branching foglia
- Lungo ogni cammino dalla radice a una foglia vengono imposti esattamente $n - 1$ vincoli di branching

Osservazione 2

Il numero totale di nodi di branching in un albero branch-and-bound binario completo con $n \geq 1$ livelli è:

$$\sum_{j=0}^{n-1} 2^j = 2^n - 1.$$

Dimostrazione (1/2)

- Per induzione su n
- **Caso base.** Per $n = 1$ l'enunciato diventa

$$\sum_{j=0}^{0} 2^j = 2^1 - 1,$$

cioè $1 = 1$, che è vero

Dimostrazione (2/2)

- **Passo induttivo.** Supponiamo che l'enunciato valga per n; lo dimostriamo per $n+1$. Abbiamo:

$$\begin{aligned}\sum_{j=0}^{n} 2^j &= \sum_{j=0}^{n-1} 2^j + 2^n \\ &= 2^n - 1 + 2^n \quad \text{(per l'ipotesi induttiva)} \\ &= 2^{n+1} - 1,\end{aligned}$$

che è l'enunciato per $n+1$

□

Una progressione geometrica

- Il numero totale di nodi è la somma dei primi n termini di una progressione geometrica di ragione $r = 2$:

$$\sum_{j=0}^{n-1} r^j = \begin{cases} \dfrac{r^n - 1}{r - 1} & \text{se } r \neq 1, \\ n & \text{altrimenti.} \end{cases}$$

- Con p variabili binarie l'albero completo ha $p + 1$ livelli e $2^{p+1} - 1$ nodi: sono i criteri di potatura a rendere l'algoritmo utilizzabile in pratica

Riepilogo

- L'algoritmo B&B estrae un nodo dalla lista, fa branching su una variabile frazionaria e risolve i due figli con la sottoprocedura
- La sottoprocedura pota il figlio per inammissibilità, per bound o per integralità (aggiornando LB e la soluzione incumbent), oppure lo inserisce nella lista
- **Regola di salto**: il secondo figlio non si risolve se, dopo il primo, $\mathrm{UB}(\mathrm{LP}_k) \leq \mathrm{LB}$; i figli saltati non ricevono indice
- Un albero binario completo con n livelli ha $2^n - 1$ nodi: la potatura è essenziale

Breve riepilogo dell'algoritmo del simplesso duale

Introduzione: Breve riepilogo dell'algoritmo del simplesso duale

- Ogni nodo dell'albero è un problema LP, e risolverli tutti da zero è uno spreco
- Vediamo come un vincolo di branching diventa una nuova riga del dizionario ottimo del padre
- Il nuovo dizionario è duale ammissibile: l'algoritmo del simplesso duale lo riottimizza con poche operazioni di pivot

Riottimizzare un figlio

- Un figlio differisce dal padre per un vincolo
- Risolverlo da zero sprecherebbe il lavoro fatto per il padre
- L'algoritmo B&B parte invece dal **dizionario ottimo del padre** e lo riottimizza con l'**algoritmo del simplesso duale**

La riga della variabile di branching

- Sia x_j la variabile di branching. È in base nel dizionario ottimo di LP_k, perché il suo valore è frazionario, e la sua riga è

$$x_j = \bar{b}_j - \sum_{q \in \mathcal{N}} \bar{a}_{jq} x_q, \qquad \text{con } \bar{b}_j = \tilde{x}_j.$$

- Alla profondità d dell'albero, il vincolo di branching riceve la variabile di scarto (o di surplus) x_s, con $s = n + m + d$

Osservazione 3: la riga del vincolo di branching (1/2)

Sostituendo la riga di x_j, i vincoli di branching diventano nuove righe in base del dizionario. Il vincolo di branching verso il basso dà:

$$x_j \leq \lfloor \tilde{x}_j \rfloor \quad \Longrightarrow \quad x_s = \lfloor \tilde{x}_j \rfloor - x_j$$

$$= \left(\lfloor \tilde{x}_j \rfloor - \tilde{x}_j \right) + \sum_{q \in \mathcal{N}} \bar{a}_{jq} \, x_q .$$

Osservazione: la riga del vincolo di branching (2/2)

Il vincolo di branching verso l'alto dà:

$$x_j \geq \lfloor \tilde{x}_j \rfloor + 1 \quad \Longrightarrow \quad x_s = x_j - \lfloor \tilde{x}_j \rfloor - 1$$

$$= \left(\tilde{x}_j - \lfloor \tilde{x}_j \rfloor - 1\right) - \sum_{q \in \mathcal{N}} \bar{a}_{jq}\, x_q .$$

In entrambi i casi il valore di x_s è negativo, mentre i costi ridotti non cambiano: il nuovo dizionario è ***duale ammissibile*** *ma non primale ammissibile.*

Il punto di partenza dell'algoritmo del simplesso duale

Un dizionario duale ammissibile ma non primale ammissibile
è esattamente il punto di partenza
dell'**algoritmo del simplesso duale**.

- Usiamo le stesse regole di pivot del modulo *Programmazione lineare*, richiamate nelle due slide seguenti

Operazione di pivot duale (1/2)

- Un'**operazione di pivot duale** agisce su un dizionario

$$x_i = \bar{b}_i - \sum_{j \in \mathcal{N}} \bar{a}_{ij} x_j, \quad i \in \mathcal{B}, \qquad z = \bar{z} + \sum_{j \in \mathcal{N}} \bar{c}_j x_j,$$

  con $\bar{c}_j \leq 0$ per ogni $j \in \mathcal{N}$

a. la **riga pivot** è la variabile in base con valore negativo di indice minimo:

$$r = \min \left\{ \; i \in \mathcal{B} : \bar{b}_i < 0 \; \right\};$$

b. se $\bar{a}_{rj} \geq 0$ per ogni $j \in \mathcal{N}$, il problema LP è **inammissibile**;

Operazione di pivot duale (2/2)

c. altrimenti la **colonna pivot** x_p realizza il rapporto minimo (pareggi: indice minimo)

$$\frac{\bar{c}_p}{\bar{a}_{rp}} = s_m = \min\left\{ \frac{\bar{c}_j}{\bar{a}_{rj}} : j \in \mathcal{N},\ \bar{a}_{rj} < 0 \right\};$$

d. dopo l'operazione di pivot il valore obiettivo è $\bar{z} + s_m \bar{b}_r \leq \bar{z}$.

- L'algoritmo si ferma quando tutte le variabili in base sono non negative: il dizionario è allora ottimo
- In un nodo figlio bastano spesso una o due operazioni di pivot duali, mentre l'algoritmo del simplesso a partire dalla base delle variabili di scarto richiederebbe diverse operazioni di pivot per ogni nodo

Riepilogo

- Ogni figlio parte dal **dizionario ottimo del padre**
- Il vincolo di branching, riscritto con la riga di x_j, è una nuova riga in base con valore **negativo**; i costi ridotti non cambiano
- Il nuovo dizionario è **duale ammissibile**: l'algoritmo del simplesso duale (riga pivot di indice minimo, rapporto minimo) lo riottimizza
- Se la riga pivot non ha coefficienti negativi, il figlio è **inammissibile**

Strategie di selezione della variabile di branching

Introduzione: Strategie di selezione della variabile di branching

- Quando più variabili intere sono frazionarie, l'algoritmo deve scegliere la variabile di branching
- La scelta ha un grande impatto sulla dimensione dell'albero branch-and-bound
- Vediamo quattro strategie: lessicografica, minima e massima inammissibilità, strong branching, e le applichiamo alla radice dell'esempio dell'auto

Scegliere la variabile di branching

- La scelta della variabile di branching ha un grande impatto sulla dimensione dell'albero branch-and-bound

Idealmente, la variabile di branching dà la massima diminuzione dei bound dei figli e porta rapidamente a soluzioni ammissibili.

Tuttavia conta anche il tempo speso per sceglierla:
una regola costosa può rallentare l'intero algoritmo.

- In tutte le strategie che seguono, i pareggi si risolvono con l'**indice minimo**

Lessicografica

La strategia **lessicografica** seleziona,
tra le variabili intere con valore frazionario,
quella di indice minimo (o massimo).

- Non richiede calcoli, ma non tiene conto né della diminuzione dei bound né della ricerca di soluzioni ammissibili
- È la regola usata negli esercizi, perché rende l'esecuzione facile da riprodurre

Minima e massima inammissibilità

La strategia di **minima** (rispettivamente, **massima**) **inammissibilità**
seleziona la variabile il cui valore frazionario è il più vicino (rispettivamente, il più lontano) a un valore intero.

- Dato un nodo di branching k, per ogni variabile intera x_j con valore frazionario $\tilde{x}_j$ in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$, il **valore frazionario** della variabile è
$$f_j = \min\{\tilde{x}_j - \lfloor \tilde{x}_j \rfloor,\ \lceil \tilde{x}_j \rceil - \tilde{x}_j\}\,.$$

Le due regole di inammissibilità

- La **minima inammissibilità** cerca di ridurre il numero di variabili frazionarie nei figli, per raggiungere più in fretta soluzioni ammissibili
- La **massima inammissibilità** elimina per prime le variabili che contribuiscono di più all'inammissibilità di $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$
- Nessuna delle due regole guarda i bound dei figli

Esempio: valori frazionari

- Al nodo radice dell'esempio dell'auto abbiamo

$$\tilde{x}_1 = \frac{7}{2}, \quad \tilde{x}_2 = \frac{5}{2},$$

e i valori frazionari sono:

$$f_1 = \min\left\{\frac{7}{2} - 3,\ 4 - \frac{7}{2}\right\} = \frac{1}{2}$$

$$f_2 = \min\left\{\frac{5}{2} - 2,\ 3 - \frac{5}{2}\right\} = \frac{1}{2}$$

- I due valori sono uguali: sia la minima sia la massima inammissibilità hanno un pareggio, che si risolve con l'indice minimo, quindi entrambe le strategie selezionano x_1

Strong branching

La strategia di **strong branching** seleziona la variabile che dà la massima diminuzione dei bound dei figli.

- Dato un nodo di branching k, per ogni variabile intera x_j con valore frazionario risolviamo i due figli e calcoliamo le diminuzioni del valore ottimo

$$\delta_k^d(x_j) = z(\mathrm{LP}_k) - z(\text{figlio verso il basso}),$$

$$\delta_k^u(x_j) = z(\mathrm{LP}_k) - z(\text{figlio verso l'alto}),$$

dove la diminuzione vale $+\infty$ per un figlio inammissibile

La variabile dello strong branching

- La variabile di branching è x_s, con

$$s = \arg\max \left\{ \min \left\{ \delta_k^d(x_j), \delta_k^u(x_j) \right\} : \right.$$
$$\left. j \in \{1, 2, \ldots, p\},\ \tilde{x}_j \text{ frazionario} \right\}.$$

- Entrambi i figli della variabile selezionata hanno una grande diminuzione, quindi entrambi hanno più probabilità di essere potati per bound

- Lo strong branching è costoso, perché risolve due problemi LP per ogni variabile candidata

- Per questo di solito si applica solo ai livelli più alti dell'albero

Esempio: strong branching (1/2)

- Al nodo radice dell'esempio dell'auto abbiamo

$$\tilde{x}_1 = \frac{7}{2}, \quad \tilde{x}_2 = \frac{5}{2} \qquad \text{e} \qquad z(\mathrm{LP}_1) = \frac{31}{2}$$

- Per ogni variabile frazionaria risolviamo i due problemi LP figli (con l'algoritmo del simplesso duale, a partire dal dizionario ottimo di LP_1)

- I risultati sono nella tabella della slide seguente

Esempio: strong branching (2/2)

Variabile	Verso il basso	z	$\delta_1^d(x_j)$	Verso l'alto	z	$\delta_1^u(x_j)$	$\min\{\delta_1^d, \delta_1^u\}$
x_1	$x_1 \leq 3$	$\frac{100}{7}$	$\frac{17}{14}$	$x_1 \geq 4$	$\frac{29}{2}$	1	1
x_2	$x_2 \leq 2$	$\frac{151}{10}$	$\frac{2}{5}$	$x_2 \geq 3$	$\frac{45}{4}$	$\frac{17}{4}$	$\frac{2}{5}$

- La massima diminuzione minima è 1: lo strong branching seleziona x_1, la stessa variabile scelta dalla regola dell'indice minimo

Altre strategie

- I solver stimano le diminuzioni δ_k^d e δ_k^u dalla storia della ricerca, senza risolvere i figli, per mezzo degli **pseudocosti**
- Le stesse stime danno una regola di selezione del nodo di branching detta **best estimate**
- Queste strategie sono una possibile estensione di questo capitolo

Riepilogo

- **Lessicografica**: indice minimo (o massimo); nessun calcolo, facile da riprodurre
- **Minima / massima inammissibilità**: valore frazionario f_j più vicino / più lontano da un intero
- **Strong branching**: massimo $\min\{\delta_k^d, \delta_k^u\}$; costoso, usato ai livelli più alti
- Alla radice dell'esempio dell'auto tutte selezionano x_1 (pareggi risolti con l'indice minimo)

Strategie di selezione del nodo di branching

Introduzione: Strategie di selezione del nodo di branching

- L'ordine in cui si elaborano i nodi della lista L è la **strategia di selezione del nodo di branching**: decide dove prosegue la ricerca
- Vediamo tre strategie: best bound, LIFO e FIFO
- Su un albero completo confrontiamo l'ordine in cui LIFO e FIFO visitano i nodi

Best bound

La strategia **best bound** seleziona il nodo della lista L con il massimo valore ottimo $z(\mathrm{LP}_k)$.

I pareggi si risolvono con l'indice di nodo minimo.

- Il nodo selezionato dà l'upper bound $\mathrm{UB} = \max_{k \in L} \mathrm{UB}(\mathrm{LP}_k)$: elaborarlo è l'unico modo per far scendere UB
- La regola tende a esplorare pochi nodi, ma la lista può diventare lunga

LIFO

La strategia **Last-In-First-Out** (LIFO)
elabora i nodi nell'ordine inverso di inserimento nella lista L.

È equivalente a una **ricerca in profondità** (DFS).

- La lista è realizzata come una **pila**
- La ricerca scende in profondità nell'albero, dove le foglie contengono spesso soluzioni ammissibili: lo scopo è aggiornare rapidamente il lower bound e la soluzione incumbent
- La lista resta corta

FIFO

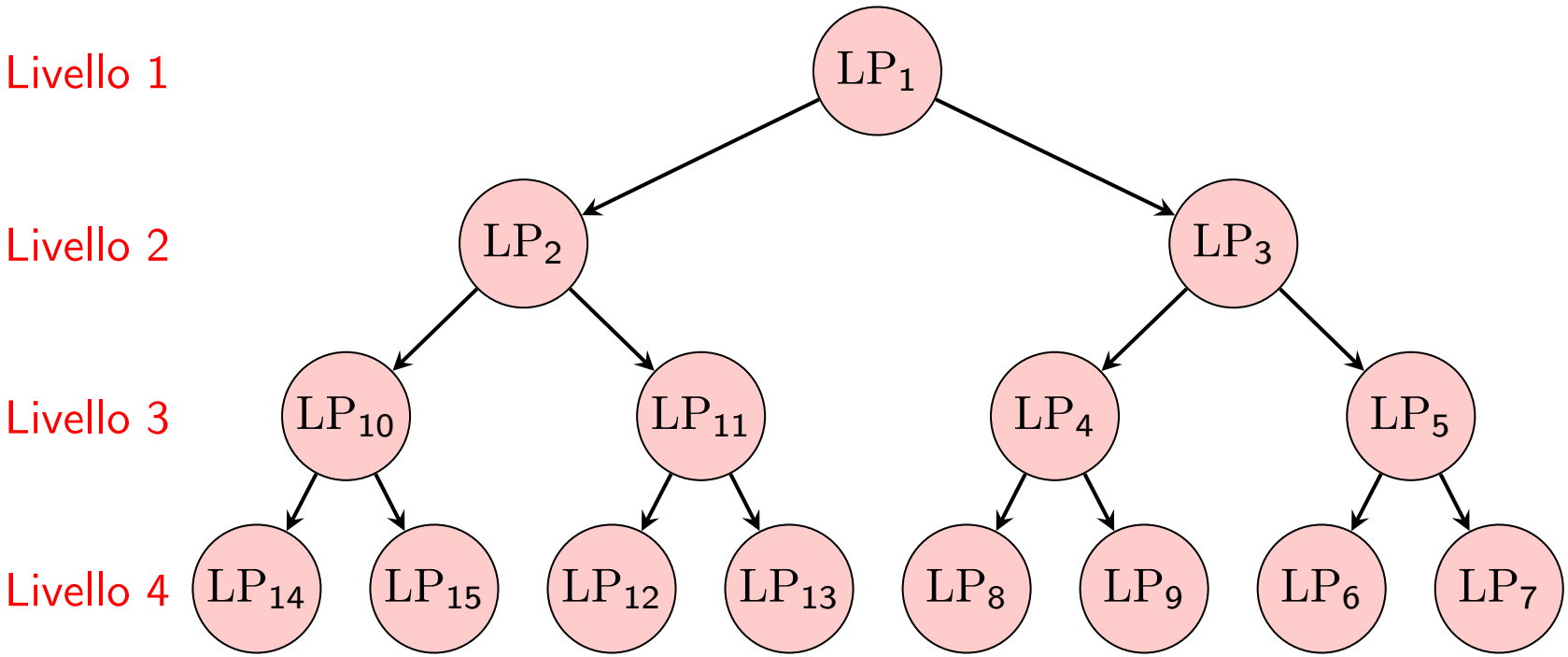
La strategia **First-In-First-Out** (FIFO)
elabora i nodi nell'ordine di inserimento nella lista L.

È equivalente a una **ricerca in ampiezza** (BFS).

- La lista è realizzata come una **coda**
- L'albero viene esplorato livello per livello: questo migliora l'upper bound, ma di solito non trova rapidamente buoni lower bound

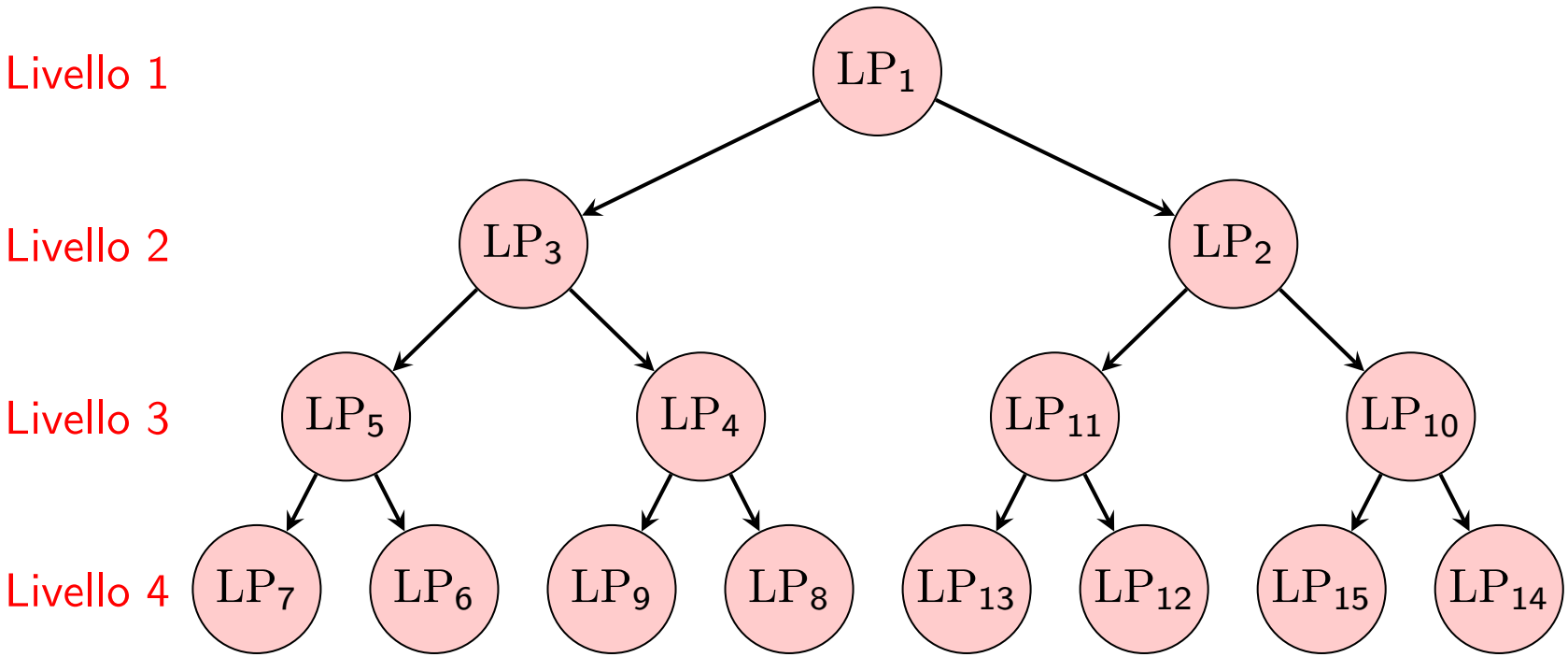
Esempio: strategia LIFO (1/2)

Ordine in cui un albero branch-and-bound binario completo di 4 livelli viene esplorato con la strategia LIFO (nessun nodo viene potato). Se si risolve per primo il nodo di branching verso il basso, abbiamo:



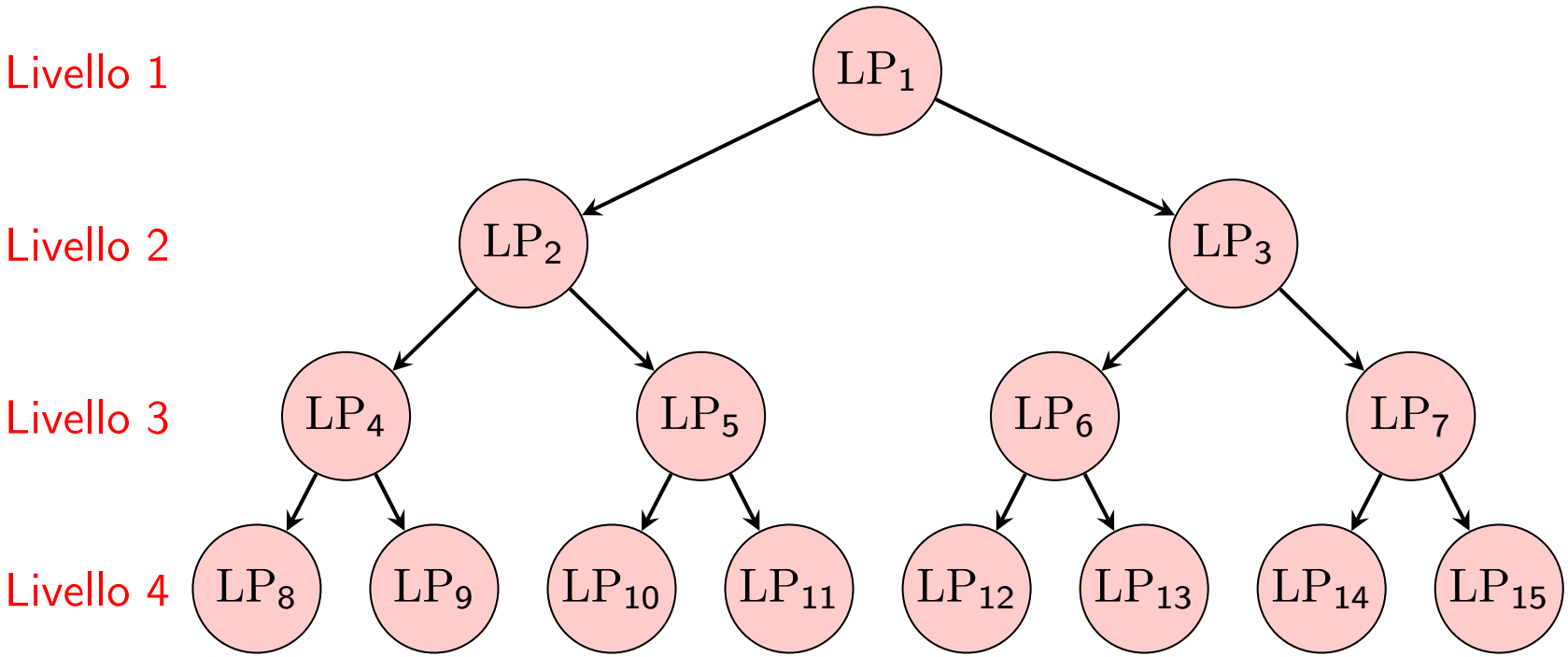
Esempio: strategia LIFO (2/2)

Se si risolve per primo il nodo di branching verso l'alto, abbiamo:



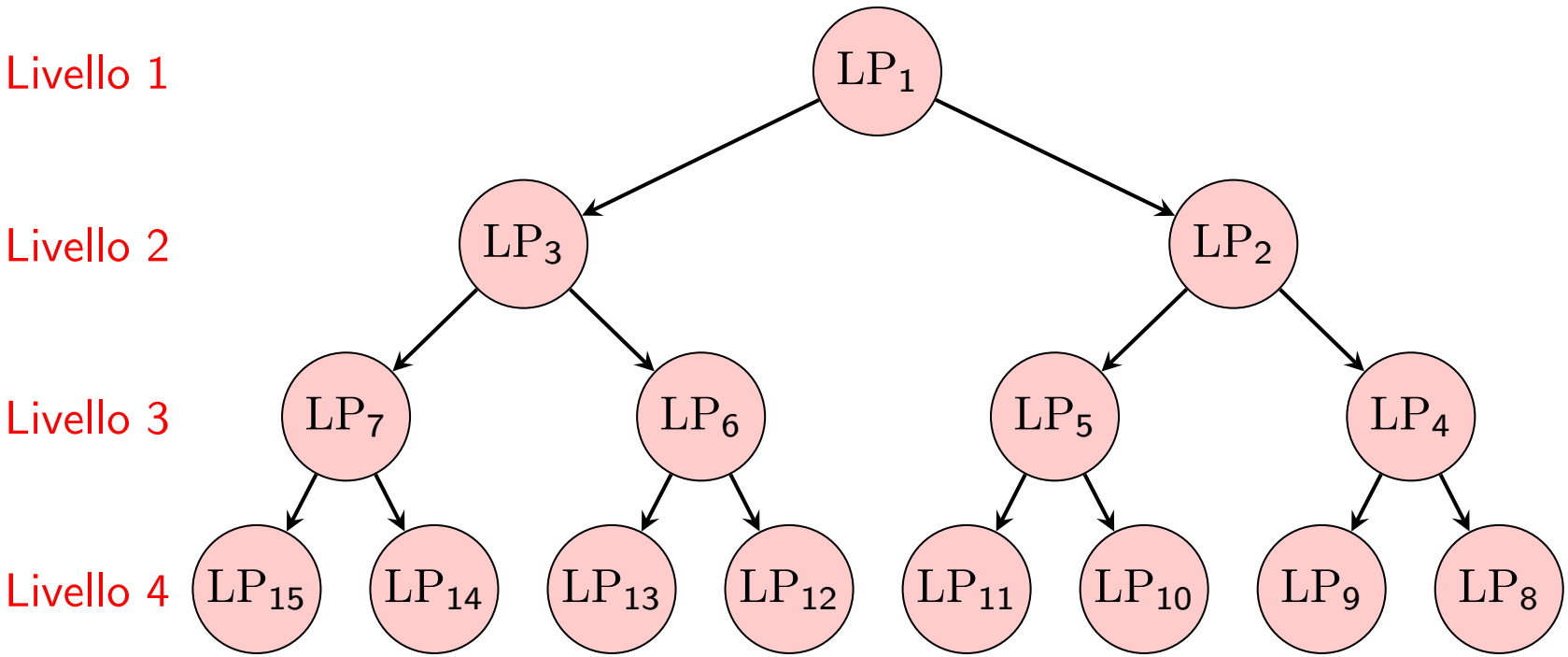
Esempio: strategia FIFO (1/2)

Ordine in cui un albero branch-and-bound binario completo di 4 livelli viene esplorato con la strategia FIFO (nessun nodo viene potato). Se si risolve per primo il nodo di branching verso il basso, abbiamo:



Esempio: strategia FIFO (2/2)

Se si risolve per primo il nodo di branching verso l'alto, abbiamo:



Riepilogo

- **Best bound**: il nodo con il massimo $z(\mathrm{LP}_k)$; l'unico modo per far scendere UB, pochi nodi ma una lista lunga
- **LIFO**: una pila, ricerca in profondità; trova presto soluzioni ammissibili, lista corta
- **FIFO**: una coda, ricerca in ampiezza; migliora l'upper bound, ma trova lentamente buoni lower bound
- L'ordine dei due figli (prima verso il basso o verso l'alto) cambia la numerazione della visita

Riepilogo della prima parte

- L'algoritmo B&B divide la regione ammissibile con vincoli di branching su variabili intere frazionarie, e non perde mai una soluzione ammissibile
- Ogni nodo è potato per inammissibilità, per integralità o per bound, oppure attende nella lista L; il bound di un nodo è $\lfloor z(\mathrm{LP}_k) \rfloor$ quando tutte le variabili e tutti i costi sono interi
- LB viene dalla soluzione incumbent, UB dai nodi della lista; la regola di salto evita figli inutili
- Ogni figlio è riottimizzato con l'algoritmo del simplesso duale a partire dal dizionario del padre
- Nella seconda parte: le esecuzioni complete dell'algoritmo sull'esempio dell'auto